

Universidad Complutense de Madrid
Máster en Ciberseguridad

Trabajo de Fin de Máster

MalWhere

Una pipeline automatizada y auditable para la
extracción de inteligencia de amenazas a partir de muestras de malware

Autor: Martín López Aramburu

Tutores: Prof. Javier Domínguez Gómez
Prof. Román Ramírez Giménez

Curso académico 2025–2026

Índice

Resumen	1
1. Introducción	2
1.1. Motivación	2
1.2. Planteamiento del problema	2
1.3. Contribuciones	2
1.4. Estructura del documento	3
2. Trabajo relacionado	4
2.1. Análisis dinámico basado en sandbox	4
2.2. Mapeo estático a ATT&CK	4
2.3. Extracción automática desde informes de amenazas	4
2.4. Agregadores de reputación y escáneres multimotor	5
2.5. Estándares de intercambio de inteligencia de amenazas	5
2.6. Posicionamiento	5
3. Arquitectura del sistema	6
3.1. Visión general de la pipeline	6
3.2. Motor de análisis estático	6
3.3. Motor de análisis dinámico	7
3.4. Reconciliación de confianza	8
3.4.1. Extensión entre niveles	8
3.5. El bucle de reenvío	8
3.6. Ampliación de la cobertura de reglas de detección	9
3.7. Despliegue y reproducibilidad	10
4. Evaluación	12
4.1. Metodología	12
4.2. Resultados	12
4.3. La auditoría de falsos positivos	13
4.4. Calibración de confianza	14
5. Casos de estudio	16
5.1. La cadena multietapa de RoningLoader	16
5.2. El error de agregación del instalador	16
5.3. El hueco de datos de referencia de WhiteSnakeStealer	16
5.4. La cascada de falsos positivos de la clave XOR	17
5.5. Contaminación de IOC de red: ruido de análisis y tráfico de fondo del sandbox	17
6. Limitaciones	19
7. Trabajo futuro	21
8. Conclusión	22
Referencias	23
A. Ampliación de cobertura de reglas de detección	24

Resumen

Los equipos de operaciones de seguridad dependen de inteligencia de amenazas estructurada (IOCs, mapeos a MITRE ATT&CK, artefactos listos para STIX/MISP) para actuar rápidamente sobre los hallazgos de malware. Los agregadores de reputación amplios como VirusTotal entregan esa estructura a una escala enorme, pero no reconcilian sus muchas señales entre sí, de modo que producir un mapeo de técnica defendible y ligado a evidencia para una muestra concreta todavía suele recaer en un analista de ingeniería inversa que traza el código manualmente y redacta un informe. Esta tesis presenta **MalWhere**, una pipeline modular que combina ingeniería inversa estática y detonación dinámica en sandbox, reconcilia sus hallazgos en mapeos de técnicas MITRE ATT&CK con confianza cuantificada, y exporta el resultado como paquetes STIX 2.1 y eventos MISP en vivo.

La pipeline se valida frente a tres familias de malware con arquitecturas distintas: un binario de ransomware (Akira); un cargador multietapa que suelta un driver rootkit, un módulo antiantivirus y un cliente C2 de acceso remoto (RoningLoader); y un infostealer .NET rico en funcionalidades (WhiteSnakeStealer). Cada una cuenta con datos de referencia obtenidos mediante ingeniería inversa manual a nivel de función, verificada con Ghidra, no mediante un escaneo superficial. Dos contribuciones metodológicas van más allá de una pipeline estática+dinámica convencional: un modelo de reconciliación de confianza cruzada entre fuentes *y* entre niveles de técnica, que trata una técnica observada tanto por evidencia estática como dinámica, o por una técnica padre y una subtécnica desde fuentes distintas, como una señal más fuerte que cualquiera de las dos por separado; y un *bucle de reenvío* que analiza de forma independiente cada componente que una muestra multietapa suelta o inyecta, sin atribuir erróneamente las capacidades de una carga soltada a la puntuación de confianza del binario padre.

La metodología de evaluación va más allá de la precisión y la exhaustividad en bruto: cada falso positivo encontrado durante la validación se rastreó individualmente hasta su evidencia exacta (una combinación de importaciones concreta, el campo de datos crudo de una firma de sandbox, un patrón de cadena) y se contrastó con el texto completo del informe manual correspondiente, no solo con su tabla resumen. De los 17 falsos positivos encontrados de esta forma en las tres muestras, doce eran errores genuinos de la pipeline con causa raíz identificada y corregida (diez sin más, más dos correcciones de calibración de confianza cuya evidencia genérica subyacente permanece presente, honestamente rebajada, en lugar de desaparecer), tres eran huecos en cómo se extrajeron los datos de referencia y no errores de la pipeline, y dos se dejaron abiertos deliberadamente a la espera de confirmación independiente en lugar de resolverse solo con la evidencia propia de la pipeline. La cobertura de detección estática se amplió posteriormente de unas 30 a aproximadamente 86 identificadores de técnica MITRE ATT&CK, cada adición obtenida de las propias descripciones de técnica de MITRE y verificada para no introducir nuevos hallazgos en las muestras validadas antes de confiar en ella.

La pipeline resultante alcanza puntuaciones F1 a nivel de familia de 0.93 (Akira), 0.81 (WhiteSnakeStealer) y 0.89 (RoningLoader, contando sus componentes reenviados), con precisión alta en las tres muestras. La tesis argumenta que este rastro de auditoría, cada regla ligada a evidencia verificable, cada error diagnosticado a una causa concreta, cada extensión comprobada frente a regresiones, es lo que hace que el resultado de la pipeline sea utilizable precisamente para la decisión que las señales no reconciliadas de un agregador de reputación no pueden sostener por sí solas, y es la propiedad con mayor probabilidad de transferirse a familias de malware que la pipeline todavía no ha visto.

Palabras clave: análisis de malware, inteligencia de amenazas, MITRE ATT&CK, STIX, MISP, análisis estático, análisis dinámico, detonación en sandbox, ingeniería inversa, reconciliación de confianza.

1. Introducción

1.1. Motivación

Cuando una muestra de malware llega a un equipo de respuesta a incidentes o de inteligencia de amenazas, el resultado útil rara vez es el propio binario: es un resumen estructurado y accionable de qué indicadores de compromiso (IOCs) presenta, qué técnicas de MITRE ATT&CK [1] implementa, y artefactos listos para alimentar ingeniería de detección (reglas Sigma, firmas YARA) o una plataforma de inteligencia de amenazas como MISP [2]. A menudo ya existe una primera pasada rápida sobre ese resumen: una consulta de hash a un agregador de reputación como VirusTotal devuelve veredictos de proveedores y etiquetas de comportamiento de sandbox en segundos. Lo que esa primera pasada no ofrece es una respuesta *reconciliada*, ya que las muchas señales que muestra se presentan una junto a otra en lugar de contrastarse entre sí, de modo que convertirlas en un mapeo de técnica defendible para una muestra concreta todavía suele significar que un analista humano detona la muestra manualmente en un sandbox, traza su código en un desensamblador y redacta el informe a mano. Ese proceso es riguroso pero lento, y no escala al volumen de muestras que un equipo de operaciones de seguridad encuentra realmente.

Los sandboxes automatizados como CAPE [3] ya producen grandes volúmenes de datos de comportamiento en bruto: trazas de llamadas a API, ficheros soltados, actividad de red, coincidencias de firmas mantenidas por la comunidad. Las herramientas de análisis estático extraen importaciones, cadenas y artefactos de cargas empaquetadas directamente del binario. Lo que generalmente falta es la capa que se sitúa entre estos hallazgos en bruto y algo que un analista o una plataforma consumidora pueda usar directamente: reconciliar evidencia estática y dinámica para la *misma* afirmación, mapear esa evidencia sobre una taxonomía de técnicas estándar con un nivel de confianza defendible, y exportarla en un formato que las plataformas de inteligencia de amenazas ya consumen.

1.2. Planteamiento del problema

Tres problemas concretos motivan este trabajo, cada uno identificado al construir y validar la pipeline descrita en esta tesis:

1. **Reconciliación de confianza entre fuentes.** Una técnica observada tanto por análisis estático (una combinación de importaciones) como por análisis dinámico (una firma de sandbox) es evidencia más sólida que cualquiera de las dos por separado, pero solo si ambas observaciones se refieren realmente a la misma técnica. Una implementación ingenua que compara observaciones por identificador de técnica exacto pasa por alto el caso habitual en el que una fuente reporta una técnica padre y la otra reporta una subtécnica específica para lo que, en realidad, es el mismo comportamiento.
2. **Malware multietapa.** Un cargador que suelta un driver rootkit, un módulo antiantivirus y un cliente de acceso remoto tiene capacidades reales distribuidas entre varios binarios, no concentradas en el único fichero detonado inicialmente. Fusionar ingenuamente la evidencia de cada componente soltado en la puntuación de confianza propia de la muestra padre atribuiría erróneamente esa capacidad, ya que el comportamiento de la carga soltada no demuestra que el cargador realice ese comportamiento.
3. **Confiar en el resultado.** Una pipeline que reporta alta confianza para un hallazgo equivocado es peor que una que no reporta nada, porque engaña activamente a quien consume el resultado. Esto motiva tratar cada regla de detección como una afirmación que necesita evidencia, y cada resultado de evaluación como algo que auditar antes de confiar en él, no solo como una cifra que reportar.

1.3. Contribuciones

Esta tesis aporta las siguientes contribuciones, cada una sustanciada en detalle en secciones posteriores:

- Un **modelo de reconciliación de confianza cruzada entre fuentes y entre niveles de técnica** (§3.4) que extiende la regla estándar “ambas fuentes coinciden $\Rightarrow$ confianza alta” para reconocer también la concordancia entre una técnica padre reportada por una fuente y su subtécnica específica reportada por la otra, un hueco confirmado en datos reales de evaluación, no meramente teórico.
- Un **bucle de reenvío** (§3.5) que analiza de forma independiente cada fichero que una muestra multietapa suelta o inyecta durante la detonación, etiquetando cada uno con su linaje hacia la muestra padre sin mezclar su evidencia en la salida de confianza propia de la muestra padre.
- Un **conjunto de reglas de detección estática reproducible y trazable a evidencia** (§3.6), ampliado sistemáticamente de unas 30 a aproximadamente 86 identificadores de técnica MITRE ATT&CK relevantes para Windows, con cada adición trazada a la descripción oficial de MITRE para esa técnica y verificada frente al conjunto de muestras validadas antes de confiar en que no sobreajusta.
- Una **metodología de evaluación y proceso de auditoría** (§4) que va más allá de precisión/exhaustividad: precisión estratificada por nivel de confianza y por acuerdo entre fuentes para comprobar si las propias afirmaciones del modelo de reconciliación se sostienen, y una auditoría sistemática de falsos positivos en la que cada discrepancia entre la salida de la pipeline y los datos de referencia verificados manualmente se diagnosticó individualmente a una causa concreta y verificable antes de corregirse, dejarse abierta, o atribuirse a un hueco en los datos de referencia.

1.4. Estructura del documento

La Sección 2 sitúa este trabajo respecto a las herramientas de sandboxing existentes, los enfoques de mapeo estático a ATT&CK, y agregadores de reputación como VirusTotal. La Sección 3 describe la arquitectura de la pipeline de principio a fin. La Sección 4 presenta la metodología de evaluación y los resultados, incluida la auditoría de falsos positivos. La Sección 5 detalla cinco casos concretos, incluyendo un error arquitectónico real encontrado y corregido a través del proceso de evaluación, que ilustran la disciplina de validación del proyecto con más concreción de lo que permiten las cifras agregadas. La Sección 6 enuncia con precisión las limitaciones de la pipeline, cada una ligada a una causa específica diagnosticada. La Sección 7 esboza extensiones concretas, incluyendo una, análisis dinámico Linux/ELF para familias de botnets IoT, evaluada y deliberadamente aplazada por razones explícitas, no implícitas. La Sección 8 concluye. El Anexo A tabula el conjunto de reglas de detección añadidas durante la ampliación de cobertura. El código fuente completo de la pipeline se publica en el repositorio público del proyecto en lugar de reproducirse aquí.

2. Trabajo relacionado

2.1. Análisis dinámico basado en sandbox

Los sandboxes automatizados de malware se remontan a Cuckoo Sandbox [4], cuya arquitectura (un controlador que orquesta la detonación dentro de una máquina virtual desechable, con un agente en el huésped que reporta llamadas a API y artefactos de vuelta al host) sigue siendo el patrón estándar. CAPE [3] (Config And Payload Extraction) bifurcó este linaje específicamente para añadir desempaquetado automático y extracción de configuración para familias de malware conocidas, e incluye una amplia biblioteca de firmas de comportamiento mantenida por la comunidad, cada una opcionalmente etiquetada con identificadores de técnica MITRE ATT&CK. Esta pipeline se apoya directamente en CAPE para el análisis dinámico (§3.3), y trata sus mapeos de firma a técnica mantenidos por la comunidad como una hipótesis de partida a verificar, no como datos de referencia a asumir; una distinción sustanciada de forma concreta en §4: varios de los mapeos propios de CAPE se comprobaron frente a su evidencia en bruto y resultaron ser sistemáticamente demasiado amplios para las muestras en las que se activaron, un paso de inspección que esta pipeline realiza y que la mayoría de despliegues de sandbox omiten.

2.2. Mapeo estático a ATT&CK

Mapear artefactos estáticos (importaciones, cadenas, coincidencias YARA) directamente a técnicas ATT&CK es una práctica común en productos EDR comerciales y en herramientas de la comunidad. El equivalente de código abierto más cercano es capa, de Mandiant [5], que compara reglas curadas de importaciones, cadenas y patrones de bytes contra un binario para reportar las capacidades que implementa, muchas de ellas ya mapeadas a ATT&CK. La mayoría de reglas de capa, como la mayoría de búsquedas de indicador de un EDR, son coincidencias de un único indicador: basta un nombre de importación o un patrón de cadena para activar la regla. Este proyecto diverge deliberadamente de ese patrón en su modelo de confianza (§3.2): un único indicador genérico (una importación usada para muchos propósitos legítimos) se trata como evidencia más débil que una *combinación coherente* de indicadores que solo tiene sentido en conjunto para la técnica afirmada. Concretamente, `OpenProcess` por sí sola no es evidencia de inyección de proceso (T1055), pero `OpenProcess` junto con `VirtualAllocEx` y `WriteProcessMemory` sí lo es, porque las tres API componen una primitiva reconocible que no tiene otro propósito común. Esto sacrifica parte de la amplitud de capa (que incluye cientos de reglas cubriendo muchas plataformas y clases de capacidad) a cambio de una precisión más estrecha pero respaldada por evidencia en las técnicas de Windows que esta pipeline sí cubre, una decisión de compromiso explicitada y medida directamente en §4.3.

2.3. Extracción automática desde informes de amenazas

Una línea de trabajo paralela extrae técnicas ATT&CK no de un binario sino de prosa de analista sin estructurar: TTPDrill [6] analiza informes de inteligencia de amenazas con una ontología propia y una pipeline de PLN para recuperar acciones del atacante; TRAM, de MITRE Engenuity [7], y rcATT [8] entrenan ambos clasificadores de texto para etiquetar párrafos de informes CTI con etiquetas de técnica ATT&CK probables. Estas herramientas resuelven un problema genuinamente distinto: ayudan a un analista que ya dispone de un *informe escrito* a convertirlo en etiquetas de técnica estructuradas y legibles por máquina de forma más rápida. Esta pipeline, en cambio, deriva sus etiquetas de técnica directamente del propio binario y comportamiento en tiempo de ejecución de la muestra, sin ningún informe como paso intermedio, lo que permite que cada hallazgo lleve asociada una pieza concreta de evidencia verificable (una combinación de importaciones, el campo de datos en bruto de una firma) en lugar de la puntuación de confianza de un clasificador de PLN sobre texto libre. Los dos enfoques son complementarios, no competidores: un clasificador al estilo TRAM podría en principio tomar como texto de entrada los propios casos de estudio escritos por esta pipeline (§5), cerrando el ciclo de evidencia binaria a informe y de vuelta a etiquetas de técnica reextraídas.

2.4. Agregadores de reputación y escáneres multimotor

VirusTotal [9] es la herramienta a la que primero recurre la mayoría de analistas de respuesta a incidentes, y constituye un punto de comparación útil porque opera en el extremo opuesto del compromiso entre amplitud y profundidad que asume esta pipeline. VirusTotal agrega veredictos de decenas de motores antivirus e integraciones de sandbox sobre un corpus de envíos de una escala que ninguna pipeline de investigación individual puede igualar, lo que lo hace excelente para el triaje rápido de reputación: si un hash ya es conocido, y si otros proveedores ya lo marcan. Lo que VirusTotal no hace es reconciliar esas señales entre sí para una única afirmación: el veredicto de cada motor y las etiquetas de comportamiento de cada sandbox se muestran uno junto a otro, no se contrastan, de modo que dos etiquetas de técnica contradictorias procedentes de dos integraciones distintas se muestran ambas sin más, sin ningún mecanismo que resuelva cuál de ellas respalda realmente la evidencia en bruto. Esta pipeline apunta precisamente a ese hueco, no a la amplitud de VirusTotal: muchas menos muestras, pero cada afirmación de técnica trazada hasta la combinación de importaciones, el campo de datos de firma o el patrón de cadena exacto que la produjo, la concordancia entre fuentes y entre niveles modelada explícitamente (§3.4), y los componentes soltados por una muestra multietapa analizados y atribuidos de forma individual en lugar de fundirse en un único veredicto agregado para el hash del padre (§3.5). Las dos herramientas no son sustitutas: una consulta a VirusTotal es el primer paso correcto para el triaje a escala, y esta pipeline es el siguiente paso correcto una vez que una muestra concreta necesita un mapeo de técnica defendible y ligado a evidencia, no una puntuación de reputación.

2.5. Estándares de intercambio de inteligencia de amenazas

STIX 2.1 [10] (Structured Threat Information Expression) y MISP [2] (Malware Information Sharing Platform) son los dos formatos de intercambio a los que exporta esta pipeline. STIX define un grafo de objetos tipados (indicadores, malware, patrones de ataque, relaciones) pensado para el intercambio máquina a máquina; MISP es a la vez un modelo de datos y una plataforma en ejecución que la mayoría de organizaciones ya operan. Exportar a ambos, en lugar de producir únicamente un informe legible por humanos, es lo que hace que el resultado de la pipeline sea directamente consumible por las herramientas que un equipo de operaciones de seguridad ya utiliza, sin necesidad de transcripción manual.

2.6. Posicionamiento

Esta tesis no propone una primitiva de detección novedosa: cada señal individual utilizada (una importación de API, un patrón de cadena, una firma de comportamiento de sandbox) es una técnica bien establecida en la práctica del análisis de malware, y herramientas como capa y CAPE ya exponen muchas de esas mismas señales en bruto. La contribución está en la *integración*: un modelo de reconciliación de confianza que es verificablemente más que la suma de sus fuentes (§3.4), un mecanismo para atribuir correctamente la capacidad distribuida del malware multietapa (§3.5), y, argumentada como la más defendible de las tres, una disciplina de validación en la que cada afirmación que hace la pipeline se comprueba frente a evidencia real antes de confiar en ella, documentada con suficiente especificidad como para que la comprobación misma sea reproducible (§4). Posicionada frente a los dos polos revisados arriba, esta pipeline se sitúa deliberadamente entre un agregador amplio y no reconciliado como VirusTotal y un informe manual de ingeniería inversa estrecho, de una sola muestra: automatizado y exportador de estándares como el primero, verificado por evidencia y específico por técnica como el segundo.

3. Arquitectura del sistema

3.1. Visión general de la pipeline

La Figura 1 muestra las cinco etapas de la pipeline. Una muestra se analiza estática y dinámicamente en paralelo; ambas salidas alimentan un *normalizador* que deduplica IOCs y agrupa las observaciones ATT&CK por técnica; un *mapeador* reconcilia la confianza entre fuentes y produce la lista final de técnicas; un *exportador* convierte esa lista en un paquete STIX 2.1 y/o un evento MISP en vivo. Cada etapa es una herramienta de línea de comandos independiente que opera sobre un esquema JSON bien definido, de modo que la pipeline puede ejecutarse de principio a fin o volver a ejecutar cualquier etapa de forma aislada, propiedad utilizada extensamente durante la validación: un solo cambio en una regla de detección a menudo solo requiere volver a ejecutar el mapeador, no la detonación completa en sandbox.

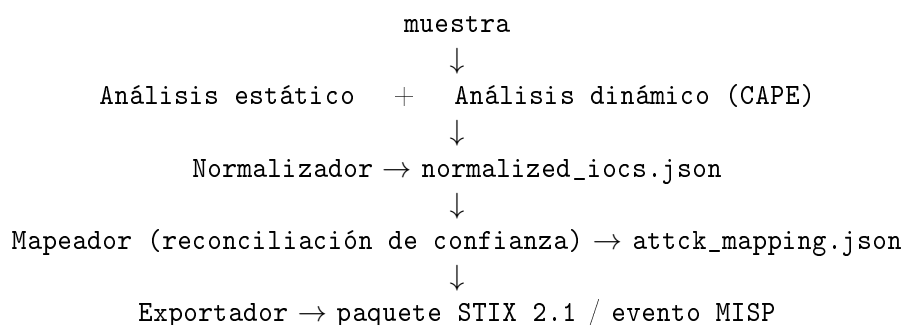


Figura 1: Etapas de la pipeline y artefactos intermedios.

3.2. Motor de análisis estático

El motor estático analiza binarios PE (nativos y .NET) y ELF, extrayendo importaciones, cadenas (estándar y decodificadas por FLOSS [11]), secciones, coincidencias YARA y características de entropía. Para muestras empaquetadas como instalador (NSIS, Inno, MSI) realiza una extracción en varias pasadas: se extrae cada fichero incluido, las claves se descubren en *todos* los ficheros extraídos antes de intentar ningún descifrado (de modo que una clave incrustada en un componente pueda descifrar una carga en otro), las claves candidatas se prueban contra cualquier fichero que parezca cifrado, y todos los ficheros, descifrados o no, se analizan completamente después. La §5.2 detalla un error real encontrado y corregido en cómo funcionaba antes esta agregación.

Las reglas de detección mapean evidencia estática a técnicas ATT&CK mediante tres mecanismos complementarios:

- **Tablas de importaciones/cadenas.** Un mapeo curado de nombres de API de Windows o patrones de cadena específicos a identificadores de técnica.
- **Comprobaciones de combinación.** Para técnicas cuyo indicador individual más fuerte también es común en software legítimo, una regla exige una *combinación coherente* específica de indicadores antes de reportar confianza elevada. La Tabla 1 enumera todas las comprobaciones de combinación implementadas mediante coincidencia explícita de múltiples importaciones; el mismo mecanismo de recuento de corroboración también rige varias reglas de patrones de cadena añadidas durante la ampliación de cobertura (p. ej. el `mshta.exe` + `.hta` de Mshta, o los tres nombres de clase corroborantes de WMI Event Subscription), tabuladas en su lugar en el Apéndice A.
- **Llamadas a la BCL de .NET.** La tabla de importaciones nativas de un binario .NET suele ser únicamente el stub de arranque del CLR, así que los cuerpos de métodos gestionados se escanean directamente en busca de llamadas a APIs específicas de la Base Class Library (p. ej. `Convert::FromBase64String`).

Técnica	Combinación requerida	Rechazado en solitario
T1055 (Process Injection)	VirtualAllocEx + WriteProcessMemory + CreateRemoteThread, o el trío de lectura OpenProcess + VirtualQueryEx + ReadProcessMemory	OpenProcess en solitario
T1055.012 (Process Hollowing)	CreateProcessW + WriteProcessMemory + SetThreadContext + ResumeThread + Nt/ZwUnmapViewOfSection	cualquier API individual
T1685 (Disable or Modify Tools)	WTSEnumerateProcessesW + OpenProcess + TerminateProcess	TerminateProcess sola
T1010 (Application Window Discovery)	EnumWindows + GetWindowTextW	GetWindowTextW sola
T1134.001/.003 (Impersonation/Theft; Make and Impersonate Token)	ImpersonateLoggedOnUser + DuplicateTokenEx (.001) o LogonUserW (.003)	Impersonate-LoggedOnUser sola

Cuadro 1: Reglas de detección sensibles a combinación. Un componente listado como “rechazado en solitario” se excluye deliberadamente de la promoción de confianza por importación única porque tiene un uso legítimo sustancial fuera de la técnica afirmada.

3.3. Motor de análisis dinámico

La detonación dinámica se ejecuta dentro de CAPE [3] contra una red simulada con INetSim, aislada y sin salida real a Internet, un diseño de contención prioritaria cuya consecuencia para la observabilidad se discute en §6. El informe en bruto de CAPE se depura en un JSON más pequeño, centrado en IOCs: firmas, ficheros soltados, actividad de red y una lista curada de mapeos ATT&CK.

Las firmas de la comunidad de CAPE llevan sus propias etiquetas de técnica, pero se trataron como una hipótesis, no como datos de referencia, durante toda la validación. La capa de curación mantiene tres tablas de corrección, cada entrada justificada individualmente frente a la evidencia en bruto de la propia firma (su campo `data`, no solo su nombre o categoría):

- **Firmas poco fiables**, descartadas por completo: p. ej. `unbacked_process_mitigation_alteration` (etiqueta T1562), cuyas direcciones de llamada en bruto en la muestra que la activó se sitúan todas en un rango estrecho de memoria alta consistente con una DLL del sistema, lo que sugiere que la clasificación de “memoria sin respaldo” falló sobre la propia inicialización automática de procesos de Windows en lugar de sobre la acción de la propia muestra.
- **Eliminaciones de técnica por firma**, donde una etiqueta en una firma por lo demás fiable es incorrecta: p. ej. `antiav_servicestop` etiqueta T1489 (Service Stop), pero en la muestra que la activó, sus propios datos en bruto nombran el servicio concreto detenido: el propio driver de la muestra, no el de un producto de seguridad.
- **Reasignaciones de técnica**, donde las propias definiciones de MITRE contradicen la etiqueta de la comunidad: p. ej. `unbacked_api_resolution` está etiquetada por la comunidad como T1129 (Shared Modules), que la propia página de MITRE define como un módulo respaldado por un fichero en disco; la descripción de la propia firma (“memoria asignada dinámicamente, sin respaldo”) es exactamente lo contrario, y T1620 (Reflective Code Loading) encaja en su lugar. La misma tabla también salva una diferencia de versión: las firmas de la comunidad de CAPE quedan fijadas a la versión de ATT&CK con la que se escribieron por última vez, así que las etiquetas T1562/T1562.001 y T1070.001 que dispara CAPE se reasignan a sus identidades de ATT&CK v19 (T1685, T1685.005) en el momento de la curación, con independencia de la versión propia de este proyecto (§6).

Una capacidad adicional, añadida durante la ampliación de cobertura, comprueba los `executed_commands` en bruto de CAPE frente a patrones de comandos LOLBin/reconocimien-

to conocidos que no tienen ninguna firma de comunidad correspondiente: p. ej. `netsh wlan show profiles`, que CAPE observa y registra pero no mapea por sí mismo a ninguna técnica.

3.4. Reconciliación de confianza

La regla central de reconciliación, implementada en `reconcile.py`, es: una técnica observada tanto por evidencia estática como dinámica alcanza confianza *alta*, salvo que la mejor confianza propia de ambas fuentes sea *baja*, en cuyo caso queda en *media*; una técnica observada por una sola fuente mantiene el nivel propio de esa fuente, y no puede promocionarse por volumen de evidencia de una única fuente.

3.4.1. Extensión entre niveles

Esta regla, tal como se implementó originalmente, comparaba observaciones agrupadas por identificador de técnica *exacto*. En la práctica, una firma dinámica frecuentemente reporta una técnica padre (p. ej. T1547) mientras que una regla estática reporta la subtécnica específica responsable (T1547.001): el mismo mecanismo de persistencia subyacente, observado en dos niveles de especificidad distintos por dos fuentes distintas, pero que nunca se corroboran entre sí bajo un agrupamiento por coincidencia exacta. Esta es una inconsistencia real, no hipotética: el propio emparejamiento a nivel de familia del arnés de evaluación (§4) ya trata una técnica padre y su subtécnica como equivalentes a efectos de puntuación, mientras que el modelo de confianza que produce el resultado puntuado no aplicaba el mismo principio.

`apply_cross_level_corroboration()` cierra este hueco: tras la pasada normal de reconciliación por técnica, las técnicas se agrupan por identificador base (quitando cualquier sufijo de subtécnica), y para cualquier grupo donde exista evidencia estática y dinámica entre sus miembros pero a un miembro individual le falte una de las dos, su confianza se recalcula usando el mejor nivel del grupo para el lado que falta, la misma regla de “alta salvo que ambas sean bajas”, extendida entre identificadores hermanos en lugar de solo dentro de uno. Cada promoción queda registrada en el propio registro de la técnica (qué técnica hermana aportó la corroboración que faltaba), de modo que el mecanismo es auditable en lugar de mezclar evidencia entre identificadores de forma silenciosa. En las tres muestras validadas este cambio no alteró ningún valor final de confianza: cada grupo con granularidad mixta actualmente alcanzaba ya *alta* de forma independiente por la propia severidad de su fuente, o ambos miembros compartían la misma única fuente. Con todo, se construyó *antes*, no después, de la ampliación de cobertura descrita a continuación, precisamente porque las nuevas reglas estáticas específicas de subtécnica combinadas con firmas de comunidad de CAPE frecuentemente a nivel de técnica padre hacen que este hueco sea más probable en adelante, no menos.

3.5. El bucle de reenvío

La capacidad real del malware multietapa suele distribuirse entre varios binarios: RoningLoader, por sí solo, a lo largo de su cadena de infección suelta o carga reflexivamente un driver rootkit en modo kernel, una DLL cuya única función es enumerar y terminar productos de seguridad, y un cliente troyano de acceso remoto independiente (§5.1). Fusionar ingenuamente la evidencia de cada componente soltado en el propio `attck_mapping.json` de la muestra padre atribuiría mal la confianza: el modelo de `reconcile.py` asume que toda observación agrupada bajo una técnica describe el *mismo* binario, y una carga soltada no demuestra que el cargador padre presente ese comportamiento.

El bucle de reenvío, en cambio, hace pasar a cada componente soltado/extraído por su propia ejecución completamente independiente a través de la misma pipeline de análisis estático → normalización → mapeo, etiquetada con linaje explícito (hash del padre, mecanismo de descubrimiento, marca de tiempo de descubrimiento) hacia la muestra que la produjo. Tres componentes, divididos por dependencia y no por conveniencia:

1. **Escritor de manifiesto** (se ejecuta en el host, solo con biblioteca estándar): localiza los bytes de ficheros soltados en el propio almacenamiento de resultados de CAPE, los copia a

- una cola, y escribe un manifiesto de linaje por candidato, acotado por un límite configurable de artefactos.
2. **Procesador estático** (se ejecuta dentro del contenedor de análisis estático, que ya tiene las dependencias más pesadas, `pefile`, `yara-python`, FLOSS, Ghidra, que necesita el análisis de ficheros soltados): analiza cada artefacto en cola y etiqueta su informe con los datos del manifiesto de linaje. Duplicar esas dependencias en un contenedor separado y más ligero se consideró y se descartó por ser superficie de mantenimiento innecesaria sin beneficio de aislamiento.
 3. **Procesador de fusión** (se ejecuta dentro del contenedor de la pipeline, solo con biblioteca estándar): localiza cada informe etiquetado con linaje que aún no se haya normalizado ni mapeado, y lo hace pasar por la misma pipeline de reconciliación de confianza que una muestra de nivel superior, escribiendo en una ruta de salida específica del componente, nunca en el propio `attck_mapping.json` de la muestra padre, de modo que la evidencia de un componente soltado nunca pueda sobrescribir o mezclarse silenciosamente con el propio resultado puntuado del padre. Esto impone a nivel de sistema de ficheros el mismo principio de no mezcla que `reconcile.py` impone a nivel de modelo de datos.

Para RoningLoader en concreto, el bucle de reenvío descubrió y analizó de forma independiente 42 componentes más allá del propio binario cargador padre: el driver rootkit, la DLL antiavivirus y el cliente RAT, cada uno con su propia entrada dedicada en los datos de referencia manuales, más otros 39 artefactos adicionales de carga y configuración extraídos sin datos de referencia propios redactados individualmente. La Sección 4 puntúa los tres componentes nombrados frente a sus datos de referencia por componente, ya que puntuar solo el componente padre subestima estructuralmente lo que encuentra la pipeline completa, incluyendo el bucle de reenvío; los 23 restantes son superficie de evaluación real adicional en un sentido distinto (§6, punto 1), cada uno confirmado como productor de un resultado sensato, sin caídas y etiquetado con linaje, en lugar de aportar nuevos puntos de datos de precisión/exhaustividad puntuados.

3.6. Ampliación de la cobertura de reglas de detección

Al inicio de la validación descrita en §4, cada regla estática existía porque evidencia específica en una de las tres muestras la justificaba. Es un enfoque sólido para evitar el sobreajuste, pero también, por construcción, deja fuera buena parte de la matriz completa de MITRE ATT&CK: cubre lo que necesitan las tres familias validadas, no la matriz en su conjunto. Para cuantificar esa cobertura con precisión en lugar de estimarla, se descargó el paquete STIX oficial actual de MITRE Enterprise ATT&CK [12] y se contó directamente: 474 técnicas relevantes para Windows (incluyendo subtécnicas) en la matriz actual, de las cuales aproximadamente 30 tenían una regla estática dedicada antes de esta ampliación.

Esas 474 técnicas ya son un subconjunto filtrado, no la matriz completa: de las 697 técnicas actuales de MITRE Enterprise ATT&CK (excluyendo las revocadas u obsoletas), solo el 68 % está etiquetado por la propia MITRE como relevante para Windows. Además, la cobertura añadida no se reparte uniformemente entre tácticas: Privilege Escalation, Persistence, Discovery, Defense Evasion, Credential Access y Execution concentran cerca del 74 % de las etiquetas táctica-técnica del conjunto de reglas, mientras que Initial Access, Lateral Movement, Reconnaissance y Resource Development, las tácticas que la acotación más abajo en esta misma sección argumenta que encajan mal con un análisis de detonación de un solo binario, no tienen ninguna regla dedicada.

La cobertura se amplió posteriormente a lo largo de seis lotes, de una a dos tácticas cada vez, añadiendo aproximadamente 55 identificadores de técnica adicionales (listado completo en el Anexo A). Cada adición sigue la misma disciplina de procedencia: el patrón de detección es el mecanismo *literal* que nombra la propia descripción de la técnica en MITRE (un valor de registro específico, una combinación de clases WMI, un marcador de fichero, una DLL, o un indicador de línea de comandos), nunca una API o herramienta genérica por sí sola cuando ese nombre también tiene usos legítimos habituales. Tras cada lote, se volvió a ejecutar el análisis

estático en las tres muestras validadas y se comprobó que no surgiera ningún hallazgo nuevo antes de considerar seguro conservar el lote; ninguna de las 55 adiciones se activó en ninguna de las tres muestras, confirmando que la ampliación no introduce nuevo riesgo de falsos positivos en el conjunto sobre el que se miden las cifras de este proyecto. Se trata explícitamente de cobertura orientada a muestras aún no vistas; no mueve, ni se afirma que mueva, las cifras de evaluación reportadas en §4.

La ampliación de cobertura se acotó deliberadamente, no de forma exhaustiva, tras comprobar el número real de técnicas para cada táctica restante de MITRE (§7): Reconnaissance y Resource Development quedan estructuralmente fuera de alcance sin importar el esfuerzo, ya que ambas describen actividad del atacante que ocurre antes de que la muestra de malware exista; Initial Access y Lateral Movement están técnicamente dentro de alcance pero encajan mal con lo que puede observar un análisis de detonación de un solo binario: para una familia de ransomware como Akira en concreto, el movimiento lateral real lo suele ejecutar el operador humano con herramientas genéricas (RDP, PsExec, Cobalt Strike) entre el acceso inicial y el despliegue del propio binario de ransomware, de modo que la capacidad normalmente no está codificada en la muestra detonada en absoluto, un mal encaje que ningún banco de pruebas de sandbox mayor cambiaría.

3.7. Despliegue y reproducibilidad

El prototipo y cada entorno de análisis del que depende, no solo el código propio de la pipeline, está definido en un único `docker-compose.yml`, dividido en dos perfiles activables de forma independiente. El perfil `core` (motor de análisis estático, pipeline, ATT&CK Navigator) no necesita configuración a nivel de host más allá de Docker en sí, y es el que produjo todos los resultados solo-estáticos de esta tesis. El perfil `sandbox` (CAPE, MISP y sus servicios de apoyo: MySQL, Redis, INetSim) necesita además una configuración manual única del host (libvirt/KVM y una máquina virtual Windows invitada instantáneable), ya que la imagen de disco de una VM no puede aprovisionarse con `docker-compose`. La Tabla 2 enumera cada servicio y su perfil.

Servicio	Perfil	Función	Memoria
<code>static</code>	<code>core</code>	Motor estático PE/ELF (<code>pefile</code> , YARA, FLOSS, Ghidra headless)	3g
<code>pipeline</code>	<code>core</code>	Normalizador, mapeador, exportador (STIX/-MISP)	1g
<code>navigator</code>	<code>core</code>	Visualización de capas del ATT&CK Navigator	512m
<code>redis</code>	<code>sandbox</code>	Backend de sesión/trabajos de MISP	256m
<code>inetsim</code>	<code>sandbox</code>	Red simulada para la detonación, sin salida real	256m
<code>cape</code>	<code>sandbox</code>	Controlador del sandbox CAPE (detonación KVM/libvirt)	6g
<code>mysql</code>	<code>sandbox</code>	Almacenamiento relacional de MISP	1g
<code>misp</code>	<code>sandbox</code>	Núcleo de MISP (almacenamiento de eventos, API, datos de galaxia)	2g

Cuadro 2: Inventario de servicios por perfil, a partir de `docker-compose.yml`.

El único paso de configuración que `docker-compose` genuinamente no puede realizar es aprovisionar el hipervisor KVM/libvirt y su red virtual en el propio host, ya que `cape` e `inetsim` necesitan un `/dev/kvm` real y un puente libvirt real al que conectarse, no un namespace de contenedor. `scripts/host-prereqs.sh` automatiza este paso único y es idempotente: instala `qemu-kvm` y `libvirt`, verifica que `/dev/kvm` sea realmente utilizable, y descubre la interfaz puente, la subred y la IP de puerta de enlace que libvirt está usando en esa máquina concreta (deliberadamente no codificado de forma fija, ya que varía de host a host), escribiendo los valores descubiertos en `.env` para que `inetsim` y la propia interpolación de configuración de CAPE puedan recogerlos sin reconstruir la imagen.

Dos decisiones de diseño más en el fichero de composición se derivan directamente de la postura de contención prioritaria descrita en §3.3. Primero, los servicios de backend (análisis

estático, pipeline, MySQL, Redis) se sitúan en una red Docker con `internal: true` y sin ninguna ruta hacia el host ni hacia internet; solo MISP y el ATT&CK Navigator, los dos servicios que un humano necesita alcanzar desde un navegador, se sitúan en una segunda red no interna, manteniendo el límite de no salida en el punto más estrecho que todavía permite a un humano revisar los resultados. Segundo, CAPE e INetSim se ejecutan con red de host en lugar de en la red de backend aislada, una excepción deliberada y acotada: la detonación de CAPE basada en KVM/libvirt no tiene ningún patrón de despliegue Docker oficialmente soportado, y todo enfoque de la comunidad converge en red de host para el propio controlador del sandbox, de modo que el límite de no salida se impone una capa más abajo, en la propia configuración de red de la VM invitada, en lugar de en la capa Docker para este servicio concreto. Ejecutar ambos perfiles juntos en un host de 16GB se acerca a unos 14GB, algo que solo encaja porque la detonación es secuencial, una VM invitada cada vez, nunca concurrente.

4. Evaluación

4.1. Metodología

El orden de esta evaluación es deliberado: cada muestra se analizó primero manualmente de forma estática, y solo después se ejecutó la pipeline sobre ella.

Los datos de referencia de cada una de las tres familias se produjeron mediante ingeniería inversa manual, documentados como un informe de analista completo y nunca derivados del propio resultado de la pipeline. Esta independencia es una salvaguarda metodológica deliberada, no un detalle incidental: ambos motores dependen de herramientas de terceros (YARA, FLOSS, firmas de la comunidad de CAPE) que pueden por sí mismas mapear mal una técnica, de modo que los datos de referencia tuvieron que establecerse sin consultar los propios artefactos extraídos por la pipeline, ya que de lo contrario validar la pipeline frente a ella sería circular. En concreto, el mismo protocolo se aplicó a las tres familias, en este orden, y solo el último paso llega a tocar el resultado propio de la pipeline:

1. **Desensamblado estático independiente.** Cada muestra se cargó en Ghidra y se descompiló función por función, cada función de interés renombrada para reflejar su propósito establecido, sin consultar la propia extracción de importaciones o cadenas de la pipeline.
2. **Observación dinámica independiente.** La detonación en sandbox se observó directamente (secuencia de llamadas a API, archivos soltados, actividad de red), registrando el comportamiento a partir de la traza en bruto en lugar de a través de las propias etiquetas de firma de la comunidad de CAPE, que §3.3 documenta como necesitadas de curación por sí mismas.
3. **Recuperación independiente de configuración y claves.** Cuando una muestra cifraba u ofuscaba su configuración, la clave o el algoritmo se recuperó manualmente, completamente separado de los propios módulos automatizados de fuerza bruta y recuperación XOR de la pipeline.
4. **Solo entonces, la comparación.** Se ejecutó la pipeline y se comparó su `attck_mapping.json` con la tabla de Técnicas ATT&CK del informe terminado y producido de forma independiente (la auditoría de falsos positivos, §4.3). Por construcción, cada discrepancia que encuentra es un desacuerdo entre dos artefactos producidos de forma independiente, no la pipeline comprobada frente a sus propias suposiciones.

La Tabla 3 da un ejemplo concreto por familia de evidencia que este protocolo recuperó y que la pipeline automatizada nunca toca, que es lo que convierte cada informe en una comprobación independiente y no en una reformulación de lo que la herramienta ya asume.

La tabla de Técnicas ATT&CK de cada informe se analiza después para producir un fichero de datos de referencia estructurado (`extract_ground_truth.py`), que el arnés de evaluación compara con el propio `attck_mapping.json` de la pipeline.

Se reportan dos modos de emparejamiento. El emparejamiento *estricto* exige una coincidencia exacta de identificador de técnica. El emparejamiento a *nivel de familia* trata una técnica padre y cualquiera de sus subtécnicas como equivalentes (un hallazgo de la pipeline de T1547.001 cuenta frente a una entrada de datos de referencia de T1547 y viceversa), bajo la premisa de que un analista de seguridad que decide qué hacer ante un mecanismo de persistencia se preocupa primero por *qué* familia de táctica y técnica está presente, siendo la granularidad de subtécnica un refinamiento y no un hallazgo distinto. Se calculan precisión, exhaustividad y F1 bajo ambos modos para cada muestra; la Tabla 4 reporta las cifras a nivel de familia, siendo las cifras más estrictas materialmente más bajas solo donde realmente no se distinguió una subtécnica concreta.

4.2. Resultados

Las dos filas de RoningLoader aíslan exactamente lo que aporta el bucle de reenvío (§3.5): puntuar únicamente el binario cargador frente a unos datos de referencia escritos para *toda* su cadena de infección subestima la exhaustividad por construcción, ya que las técnicas propias del

Muestra	Recuperado de forma independiente, la pipeline no lo toca
Akira	Tabla ATT&CK de 17 filas construida a partir de descompilación de Ghidra a nivel de función de los mecanismos internos de threading, logging y cifrado de la muestra.
RoningLoader	Los cuatro componentes de la cadena (instalador, D3D11InstallHelper.dll, driver rootkit, cliente RAT) descompilados por separado, incluyendo identificar DoD3D11InstallUsingMSI del auxiliar como una reimplimentación compilada en Rust de <code>env::var()</code> .
WhiteSnakeStealer	Clave RC4 recuperada mediante análisis de texto plano conocido, y el formato completo del paquete C2 WSR\$ (XML de información del sistema cifrado con RC4 y comprimido con Deflate, con una clave envuelta en RSA) reconstruido directamente a partir del desensamblado.

Cuadro 3: Un ejemplo concreto por familia de evidencia recuperada de forma independiente para construir los datos de referencia, aplicando el protocolo de cuatro pasos anterior.

Muestra	Precisión	Exhaustividad	F1	Notas
Akira (ransomware)	0.87	1.00	0.93	binario de una sola etapa
WhiteSnakeStealer (.NET)	0.97	0.70	0.81	binario de una sola etapa
RoningLoader (solo padre)	0.95	0.77	0.85	solo el cargador
RoningLoader (+reenvío)	0.86	0.92	0.89	cargador + componentes soltados

Cuadro 4: Precisión/exhaustividad/F1 a nivel de familia por muestra validada.

driver rootkit y del cliente RAT son hallazgos reales que los datos de referencia esperan pero que el binario padre por sí solo no puede presentar. La exhaustividad sube 15 puntos en cuanto se analizan de forma independiente los tres componentes soltados y se cuentan sus hallazgos, a un coste de 9 puntos de precisión por la superficie añadida, una compensación que la mejora del F1 agregado ($0.85 \rightarrow 0.89$) demuestra que mereció la pena.

La precisión es alta en las cuatro filas (0.86–0.97), es decir, el pequeño número de hallazgos que reporta la pipeline rara vez son erróneos. Esta es la consecuencia directa e intencionada de la filosofía de combinación coherente de §3.2 y la disciplina de curación de firmas de §3.3: ambas sacrifican algo de exhaustividad a cambio de alta confianza en que lo que *se* reporta es fiable, algo que la auditoría de falsos positivos a continuación existe para comprobar, no para asumir.

4.3. La auditoría de falsos positivos

La precisión por sí sola no distingue “la pipeline está bien calibrada” de “los datos de referencia resultan ser permisivos”. A diferencia de un agregador de reputación que muestra la etiqueta de cada motor o sandbox tal cual, cada discrepancia entre el resultado de la pipeline y los datos de referencia encontrados en las tres muestras, 17 en total (6 Akira, 5 WhiteSnakeStealer, 6 RoningLoader), se rastreó individualmente hasta una causa concreta y verificable en lugar de aceptarse o descartarse solo por la cifra agregada. Cada una se contrastó con el *texto completo* del informe manual correspondiente, no solo con su tabla resumen, ya que una técnica puede estar genuinamente discutida en la narrativa de un informe sin aparecer en su propia tabla ATT&CK.

El resultado de esta auditoría, por causa:

- **Diez errores genuinos de la pipeline**, cada uno corregido en su causa raíz: tres mapeos de regla estática que no correspondían con la definición real de MITRE para la técnica afirmada (p. ej. `GetSystemInfo` mapeada a Descubrimiento de Información del Sistema, T1082, cuando la propia definición de MITRE trata específicamente sobre detección de *vir-*

tualización/sandbox, T1497); un hueco en una comprobación de combinación que permitía que una única importación genérica activara T1055 por sí sola; y seis mapeos de firmas de la comunidad de CAPE cuyo propio campo de evidencia en bruto, al inspeccionarlo, nombraba un objetivo inconsistente con la técnica afirmada (detallado en §3.3).

- **Dos correcciones de calibración de confianza**, en Akira y RoningLoader: el mismo hueco de comprobación de combinación de la corrección de T1055 anterior reapareció para T1070 (Indicator Removal) – `CreateFileW/CreateFile`, ya marcada como “genérica” en la tabla de mapeo, seguía contándose ciegamente como corroboración junto a `DeleteFileW/DeleteFile` y se promovía a confianza *alta*. Migrar los datos de referencia a ATT&CK v19 (§6) es lo que lo sacó a la luz: la antigua entrada de datos de referencia T1070.001 había estado emparejándose con este hallazgo por pura coincidencia de prefijo de identificador bajo el emparejamiento a nivel de familia, no porque la evidencia tratara genuinamente sobre el asunto real de esa entrada (el borrado de registros de eventos); mover esa entrada a su identidad v19 (T1685.005, que ya no comparte el prefijo T1070) rompió la coincidencia casual. Corregido de la misma manera que el caso de T1055, excluyendo la importación/cadena genérica de la corroboración en ambas direcciones. A diferencia de aquel caso, la evidencia subyacente aquí es real (ambos binarios importan efectivamente estas funciones), simplemente no lo bastante específica para confirmar la técnica afirmada, así que el hallazgo permanece presente tras la corrección, ahora con su confianza honesta *media/baja*, en lugar de desaparecer del todo – una ilustración directa de la relación entre confianza y precisión examinada en §4.4, no un contraejemplo de ella.
- **Tres huecos de extracción de datos de referencia**, no errores de la pipeline: el propio informe manual de WhiteSnakeStealer discute la carga reflexiva de DLL en su texto narrativo, pero la fila correspondiente de T1620 faltaba en la tabla ATT&CK del propio informe; ese mismo informe también documenta que koBz enumera el software de seguridad/antivirus instalado a través del registro y que el ladrón de Thunderbird lee nueve claves de perfil en busca de credenciales almacenadas – la propia acción de leer/enumerar (T1012, Query Registry), distinta de lo ya registrado en T1112 (escrituras) y T1552.002 (el contenido robado); y el informe de Akira describe cómo `enumerate_drives()` recorre todas las unidades y clasifica cada una mediante `GetDriveTypeW/PathIsNetworkPathW` antes de cifrar, es decir, T1082 (Descubrimiento de Información del Sistema). Ninguna de las tres tablas de técnicas capturó estos hallazgos solo narrativos. En los tres casos el hallazgo de la pipeline era correcto; se corrigió el documento fuente en lugar de la pipeline.
- **Dos dejados abiertos deliberadamente**, en RoningLoader (T1027, T1497): la evidencia de apoyo de la pipeline para ambos es real, pero no existe confirmación explícita del analista en ningún lugar del texto del informe, a diferencia del caso de T1620 anterior. Editar los datos de referencia solo con la evidencia propia de la pipeline, sin confirmación textual independiente, sería circular. Ambos permanecen como hallazgos abiertos en lugar de resolverse en un sentido u otro.

Esta asimetría, tres huecos de datos de referencia corregidos y dos discrepancias de aspecto similar dejadas abiertas, es deliberada: el factor distintivo es si existe confirmación independiente en el documento fuente, no si la evidencia propia de la pipeline parece convincente. El resultado práctico de ejecutar esta auditoría se aprecia en la columna de corregidos: 12 de las 17 discrepancias se tradujeron en mejoras concretas y verificables al conjunto de reglas (§3.6) y a la lógica de agregación (§5.2) en lugar de absorberse en silencio dentro de una cifra agregada, precisamente el paso diagnóstico que una visualización de veredictos uno junto a otro no puede ofrecer.

4.4. Calibración de confianza

Si el modelo de reconciliación de confianza (§3.4) es significativo y no meramente cosmético, la precisión debería ser medible y mayor entre los hallazgos de confianza *alta* (aquellos corroborados entre fuentes o entre niveles de técnica) que entre los hallazgos de *media/baja* de una sola fuente.

En las tres muestras validadas, 16 de las 17 discrepancias identificadas en la auditoría anterior tuvieron su origen en un hallazgo de una sola fuente. La única excepción, **T1497** de RoningLoader, alcanzó confianza *alta* mediante corroboración genuína entre fuentes (una combinación coherente de APIs anti-sandbox en la evidencia estática, tres firmas independientes de CAPE en la evidencia dinámica), y sin embargo sigue siendo uno de los dos hallazgos dejados abiertos deliberadamente más arriba, precisamente porque no existe confirmación textual independiente en el informe manual, no porque la evidencia de ninguna de las dos fuentes sea débil. Leído así, esto no contradice el argumento de calibración: el modelo reconoció correctamente una evidencia fuerte y corroborada; lo que es genuinamente incierto es si los datos de referencia están completos, la misma asimetría que establece el caso de **T1620** en otro punto de esta auditoría, no si el nivel de confianza asignado estaba justificado. Esto es consistente con que el modelo de reconciliación realiza una calibración real y no simplemente reetiqueta hallazgos, aunque el tamaño reducido de la muestra (tres familias) implica que esto debe leerse como una señal direccional, no como una afirmación con potencia estadística, una limitación enunciada explícitamente en §6.

5. Casos de estudio

Las cifras agregadas (§4) muestran *que* la pipeline rinde bien; los cinco casos siguientes muestran *por qué*: instancias concretas de la disciplina de validación encontrando y resolviendo un problema real, cada una ilustrando una clase de fallo distinta que una evaluación puramente numérica no sacaría a la luz por sí sola.

5.1. La cadena multietapa de RoningLoader

La cadena de infección de RoningLoader, establecida mediante ingeniería inversa manual, no es un binario sino cuatro: un cargador que descifra y suelta un driver rootkit en modo kernel usado para ocultar su propio proceso y ficheros, una DLL independiente cuya única función es enumerar y terminar productos de seguridad, y un cliente de acceso remoto que establece el canal C2 real. La capacidad de cada componente es real y significativa por sí misma; ninguno de los tres componentes soltados aparece como importación o cadena en el propio binario cargador, ya que se descifran en memoria o se sueltan a disco únicamente en tiempo de ejecución.

La Tabla 4 ya muestra la consecuencia numérica: puntuar el cargador de forma aislada alcanza una exhaustividad de 0.68 frente a unos datos de referencia escritos para la cadena completa, porque aproximadamente un tercio de las técnicas que un analista humano atribuye correctamente a la familia “RoningLoader” solo son observables analizando directamente los componentes soltados. El bucle de reenvío (§3.5) existe precisamente porque esto es una propiedad estructural del malware multietapa, no una peculiaridad de una muestra, y cierra 16 de esos 32 puntos de exhaustividad al dar a cada componente su propio análisis independiente en lugar de intentar inferir su capacidad a partir de la evidencia del padre.

5.2. El error de agregación del instalador

Al principio de la validación, el informe estático de una muestra empaquetada como instalador mostraba evidencia consistente solo con *uno* de sus varios componentes incluidos, pese a que la lógica de extracción en varias pasadas (§3.2) había procesado todos ellos. Al trazar directamente el código de generación del informe, sin asumir que el fallo estaba en la propia lógica de extracción, se encontró el defecto real una etapa después: `_analyze_installer`, la función que agrega los resultados por componente en el informe único de nivel superior de la muestra, estaba *sustituyendo* silenciosamente el análisis de un componente hijo por el de otro bajo una condición específica, en lugar de fusionar ambos.

El error se confirmó, no solo se infirió del síntoma, a través del propio campo `hash_match` del informe, un valor registrado precisamente para este tipo de comprobación cruzada, que mostraba que el hash reportado pertenecía a un fichero extraído distinto del que realmente aparecía en el cuerpo del informe. Corregir la lógica de agregación para fusionar los hallazgos de cada componente extraído, en lugar de solo el procesado más recientemente, restauró la evidencia faltante y mejoró directamente la exhaustividad en la muestra afectada tras volver a evaluarla.

La lección se generalizó más allá de esta corrección concreta: motivó tratar cada nueva capacidad de la pipeline, el bucle de reenvío en primer lugar, como necesitada de su propio paso explícito de corroboración (manifiestos de linaje, comprobaciones cruzadas de hash) en lugar de confiar en que “el código se ejecutó sin error” implique “el código produjo el resultado correcto”. Encontrar y corregir esta clase de error antes de que pudiera subestimar en silencio la capacidad real de una muestra es en sí mismo una demostración concreta de para qué sirve la disciplina de validación de la pipeline.

5.3. El hueco de datos de referencia de WhiteSnakeStealer

La §4.3 resume el resultado; el proceso merece detallarse una vez como ejemplo concreto de lo que significa en la práctica “auditado frente al texto completo del informe”. El resultado de la pipeline para WhiteSnakeStealer incluía T1620 (Reflective Code Loading) con evidencia estática, llamadas específicas a la BCL de .NET consistentes con la carga de ensamblados en memoria.

La propia tabla de Técnicas ATT&CK de la muestra, sin embargo, no tenía fila de T1620, por lo que la evaluación automática inicialmente puntuó esto como un falso positivo.

Al leer la sección narrativa del informe, en lugar de confiar solo en su tabla resumen, se encontró un párrafo explícito que describía exactamente el mecanismo de carga reflexiva que había detectado la regla estática: existía confirmación independiente del analista, simplemente no se había trasladado a la tabla que lee el extractor automático de datos de referencia. Esto es un hueco de documentación en el informe fuente, no un error de la pipeline, y se corrigió añadiendo la fila que faltaba al propio informe (y regenerando los datos de referencia a partir de él), en lugar de ajustar la confianza de la pipeline o suprimir el hallazgo. Se presenta aquí específicamente porque dos casos estructuralmente similares en RoningLoader (§4.3) se resolvieron en el sentido opuesto, dejados abiertos, precisamente porque les faltaba este tipo de confirmación textual independiente. Juntos, los dos desenlaces muestran que la evidencia propia de la pipeline se trata igual independientemente de si la discrepancia terminó favoreciéndola.

5.4. La cascada de falsos positivos de la clave XOR

La fuerza bruta XOR de un solo byte (`config_parser.py`) prueba exhaustivamente las 256 claves posibles contra regiones candidatas de texto cifrado y compara el resultado con una expresión regular de IP en formato punteado, la técnica que recuperó la IP de C2 genuína de RoningLoader (202.95.11.173) sin conocimiento previo de la clave (§6, punto 4). Ejecutada sin salvaguardas contra una segunda muestra, WhiteSnakeStealer, la misma rutina produjo 8 coincidencias de cuarteto punteado sintácticamente válidas pero completamente espurias a partir de una única clave incorrecta (0x2e, ASCII .): una tabla de enteros repetitiva con relleno de ceros en los datos compilados del binario resultó decodificar a ocho direcciones IP falsas bajo esa única clave, ya que los bytes nulos se decodifican como caracteres . literales bajo ella y una estructura repetitiva en los datos de origen produce una coincidencia falsa repetitiva.

Dos filtros cerraron el hueco, cada uno justificado por una propiedad distinta que un valor incrustado genuino tiene y un accidente de decodificación no. Primero, una **comprobación de límite limpio**: el byte inmediatamente anterior y posterior a una coincidencia debe ser 0x00 o el propio byte de clave, algo que una cadena real incrustada y terminada en nulo cumple y que una secuencia de bytes arbitraria decodificada por una clave no relacionada generalmente no. Segundo, un **límite de densidad de coincidencias** de dos coincidencias con forma de IP por clave en cualquier lugar del fichero, bajo el principio de que un valor de configuración genuino aparece una vez, o raramente un puñado de veces, mientras que una cascada producida al decodificar datos estructurados no relacionados bajo la clave equivocada no se autolimita. Se verificó después en ambas direcciones: la IP de C2 genuína de RoningLoader sigue produciendo exactamente una coincidencia y sobrevive al límite de densidad; las ocho coincidencias espurias de WhiteSnakeStealer ahora se descartan correctamente.

La lección se generalizó más allá de esta corrección concreta: es lo que motivó la política explícita del proyecto (§6, punto 5) de que cualquier nueva capacidad de recuperación basada en búsqueda exhaustiva debe validarse frente a una segunda muestra, no solo la que motivó su construcción, antes de confiar en ella. De haberse declarado esta capacidad lista para producción solo con RoningLoader, la pipeline habría salido con un generador de falsos positivos que simplemente todavía no se había activado.

5.5. Contaminación de IOC de red: ruido de análisis y tráfico de fondo del sandbox

Auditar directamente la salida de IOC de la pipeline orientada a STIX/MISP, en lugar de solo las puntuaciones de técnicas ATT&CK que ya cubre §4, sacó a la luz dos fuentes de contaminación independientes que ninguna métrica a nivel de técnica puede ver, ya que ninguna cambia qué técnicas se reportan: corregir ambas dejó el conjunto de técnicas, la precisión y la exhaustividad de cada muestra idénticos byte a byte.

Primero, la expresión regular de cadenas punteadas del extractor estático de dominios se

filtraba solo con una lista de exclusión escrita a mano, por lo que tokenizaba erróneamente metadatos del compilador como dominios (`kernel32.dllfailed`) y, a través de los 42 componentes reenviados de `RoningLoader` (§3.5), toda una clase de endpoints de cadena de certificados `Authenticode` (`cr1.comodoca.com`) presentes en prácticamente cualquier PE firmado con código – ninguno de ellos infraestructura elegida por el adversario. Una lista de permitidos de TLD, una comprobación de mayúsculas de identificador compuesto y una pequeña lista de permitidos de coincidencia exacta para los endpoints PKI confirmados como benígnos eliminaron cada instancia en las tres familias y los 42 componentes reenviados.

Segundo, e independiente de lo anterior, la captura de red dinámica de CAPE (`network.hosts`) es un volcado PCAP de toda la máquina virtual sin atribución por proceso: no puede distinguir las conexiones propias de la muestra del tráfico de fondo del sistema operativo huésped. Cotejar cada IP observada en las detonaciones de las tres familias contra datos de registro de ASN encontró 18 que resolvían a Microsoft (telemetría de Azure AD/Teams) o a Akamai (CDN de Windows Update/Defender) y se repetían byte a byte en un ransomware, un infostealer y un cargador – tres muestras sin razón plausible para compartir infraestructura. Una lista de permitidos de alcance estrecho con estas IP específicas, confirmadas individualmente, y no una regla general de ASN/CIDR (que habría descartado también domain fronting legítimo del adversario), las eliminó de los IOC reportados.

La corrección es verificable de forma independiente frente a los propios datos de referencia de `WhiteSnakeStealer`, que documentan una lista C2 HTTP de 31 nodos y 29 IP únicas. Tras el filtro de ruido de sandbox, la salida de captura dinámica de la pipeline recupera exactamente esa lista: 29 de 29, cero omisiones, corroborado de forma independiente por la propia firma `dead_connect` de CAPE.

La contaminación no era del todo cero, sin embargo: al reauditar esta misma afirmación durante la migración a ATT&CK v19 (2026-08) se encontraron 5 IP adicionales que los dos filtros anteriores no capturaban – ninguna afecta a las puntuaciones a nivel de técnica, ya que el filtrado de IOC y la puntuación de técnicas son independientes. Cuatro resultaron ser la misma clase de ruido de sandbox que las 18 anteriores (dos nombres de host `*.msedge.net` en el ASN de Microsoft, dos direcciones de Cloudflare sin nombre de host confirmadas cruzando la traza de llamadas a `connect()` por proceso, no solo la captura PCAP resumida) y se añadieron a la lista de permitidos. La quinta, que resuelve a `ip-api.com`, resultó ser un hallazgo genuino: la propia muestra llama a `WinHttpGetProxyForUrl` contra un endpoint público de geolocalización por IP sin ninguna cadena literal correspondiente en el binario, la URL construida en tiempo de ejecución. Se añadió a los datos de referencia manuales de `WhiteSnakeStealer` como T1016 (System Network Configuration Discovery), con el mismo estándar de evidencia que cualquier otra entrada de ese informe.

La lección se generaliza igual que en los demás casos: cada exclusión es una entrada específica, justificada individualmente y auditable, registrada con su propia razón, no un filtro de reputación general, ya que la razón de ser de esta pipeline frente a un agregador de reputación (§2) es precisamente que cada decisión permanece trazable a una causa concreta en lugar de simplemente afirmarse.

La generalidad de la propia corrección se comprobó, no se asumió: ejecutar la misma disciplina frente a una cuarta familia estructuralmente distinta (un RAT/backdoor .NET envuelto en un crypter nativo, añadida específicamente para comprobar si la pipeline generaliza más allá de las tres familias validadas) sacó a la luz dos instancias más del mismo fallo en sus propios componentes reenviados, más una clase nueva no vista hasta entonces (OID de certificado X.509 confundidos con IP por el mismo extractor basado en expresiones regulares). Cada uno se corrigió de la misma forma: una regla general y justificada por evidencia, verificada frente a cada IP genuina conocida en las dos familias validadas con IOC numéricos.

6. Limitaciones

Enunciar las limitaciones con esta precisión es en sí mismo parte de la contribución argumentada en §2: los veredictos uno junto a otro de un agregador de reputación no llevan un relato equivalente de dónde podrían estar equivocados, lo que los hace rápidos de consultar pero difíciles de auditar. Cada limitación siguiente se diagnostica, en cambio, a una causa concreta y verificable en lugar de enunciarse de forma genérica, de modo que cada una ya está mitigada de forma concreta o se acota como un siguiente paso preciso (§7) en lugar de quedar como una advertencia abierta.

1. **Conjunto de validación pequeño (N=3).** Tres familias son pocas para una afirmación con potencia estadística, y es el punto que más merece escrutinio en este trabajo. El tamaño no es solo una limitación de tiempo: los directores de esta tesis recomendaron explícitamente acotarlo a tres familias para mantener el proyecto dentro de un alcance manejable como TFM, priorizando la profundidad de cada validación frente a dispersar el esfuerzo entre más muestras. Tres mitigaciones compensan parcialmente ese tamaño: los datos de referencia son ingeniería inversa manual a nivel de función, verificada con Ghidra, para cada muestra, un listón considerablemente más alto que el que tendría un conjunto más grande pero superficial; una auditoría completa de `static/scripts/` centrada en hardcoding específico de muestra disfrazado de lógica genérica encontró y eliminó cuatro instancias, cada una sustituida por un equivalente a nivel de clase y reverificada para no provocar regresión en ninguna muestra validada; y el bucle de reenvío de RoningLoader (§3.5) puntúa de forma independiente 42 componentes adicionales frente a los mismos datos de referencia, superficie de evaluación real adicional y no un truco estadístico. Ampliar a más familias sigue siendo el siguiente paso de mayor valor (§7).
2. **La versión de ATT&CK fijada exige migración manual periódica.** Todos los identificadores de técnica en esta pipeline, los datos de referencia y las capas de Navigator apuntan a MITRE ATT&CK v19.2; las firmas de la comunidad de CAPE, en cambio, quedan fijadas a la versión con la que se escribieron por última vez y se reasignan a su identidad actual solo en el momento de la curación (§3.3), no de forma automática en origen. Esta tesis migró sus propios datos de referencia de v14 a v19 durante su desarrollo, un trabajo manual no trivial que a su vez sacó a la luz el fallo de calibración de confianza de T1070/T1685.005 detallado en §4.3. Cada futura revisión de MITRE ATT&CK exigirá repetir ese mismo esfuerzo, ya que nada en el diseño actual lo automatiza.
3. **El sandboxing con contención prioritaria limita la observación de red/C2.** CAPE detona cada muestra contra la red simulada de INetSim, sin salida real, por diseño. Varias técnicas no detectadas en las tres muestras se agrupan en torno a comportamiento de exfiltración y proxy/túnel (T1041, T1090, T1571/T1572, T1046) que una red simulada no puede provocar por completo. Esto es una propiedad estructural de un entorno de investigación con contención prioritaria, no un defecto a corregir.
4. **La recuperación estática de claves/configuración tiene un techo preciso.** El XOR de un solo byte es exhaustivamente forzable por fuerza bruta (128 claves), y así se usó para recuperar la IP de C2 de RoningLoader sin conocimiento previo de la clave, pero solo tras añadir dos filtros independientes de falsos positivos (§5.4). En cambio, la clave RC4 de una carga distinta, verificada de forma independiente mediante ingeniería inversa manual con Ghidra, se ensambla en *tiempo de ejecución* a partir de tres constantes combinadas mediante aritmética de registros y nunca existe como secuencia de bytes contigua en el fichero: ningún método estático de patrón de bytes, por exhaustivo que sea, puede recuperar una clave que nunca se almacena como bytes. El límite está precisamente en si la clave existe como bytes en disco, no en una afirmación general de que “el cifrado es difícil”.
5. **Limitaciones a nivel de herramienta.** La decodificación de cadenas basada en emulación de FLOSS agotó el tiempo en Akira, la muestra más agresiva en anti-análisis; esto

se registró explícitamente en lugar de faltar en silencio. La recuperación AES/ChaCha20 solo tiene éxito frente a un nonce/IV cero. Cualquier nueva capacidad de recuperación por fuerza bruta ahora se valida, por política explícita, frente a una segunda muestra antes de confiar en ella, tras detectarse así la cascada de falsos positivos de la clave XOR (§5.4) en lugar de al primer éxito.

6. **Reproducibilidad.** La §3.7 describe el despliegue completo con `docker-compose`; el único hueco que merece repetirse aquí es que la máquina virtual Windows del perfil `sandbox` necesita una configuración manual única del host (libvirt/KVM), documentada con sus propias notas de solución de problemas, ya que no puede crearse solo con `docker-compose`.
7. **La cobertura de reglas estáticas es deliberadamente estrecha.** Aproximadamente 86 de las 474 técnicas relevantes para Windows en MITRE tienen una regla estática dedicada, con la justificación del alcance detallada en §3.6.
8. **La captura de red dinámica no tiene atribución por proceso.** La lista `network.hosts` de CAPE refleja el tráfico de toda la máquina virtual del sandbox, no solo el del propio proceso de la muestra, por lo que la telemetría de fondo del sistema operativo huésped puede aparecer indistinguible de las conexiones reales de la muestra (§5.5). La mitigación aplicada es una lista de permitidos estrecha con las IP específicas confirmadas individualmente que realmente contaminaban las detonaciones de estas tres muestras, no un filtro general de ASN o de reputación; una muestra recién detonada podría mostrar la misma clase de contaminación bajo una IP distinta que la lista actual todavía no cubre.

7. Trabajo futuro

- **Familias adicionales validadas.** La extensión de mayor valor (§6, punto 1) es una validación completa respaldada por ingeniería inversa manual de más familias. Ya se realizó una comprobación más ligera e informal frente a una 4ª familia sin afinar (AsyncRAT): una detonación real en CAPE más el reenvío automático de sus cargas soltadas confirmó que la pipeline produce un resultado sensato y sin caídas de principio a fin. Esto carece de datos de referencia manuales y no se trata como evidencia de validación, solo como una comprobación interna de cordura de que la pipeline generaliza en absoluto (documentada en README.md); no sustituye a la extensión completa con datos de referencia que describe este punto.
- **Ampliar aún más la cobertura de reglas estáticas.** Las 86 de 474 técnicas relevantes para Windows de §3.6 dejan aproximadamente 388 sin cubrir. Cerrar esa brecha es un trabajo deliberadamente lento, técnica a técnica, no un conjunto de tareas para hacer en bloque: cada adición necesita su propia justificación a nivel de descripción de MITRE y una nueva ejecución completa frente a las muestras validadas para descartar nuevos falsos positivos, el mismo listón que se aplicó a la ampliación original de 30 a 86.
- **Análisis dinámico Linux/ELF para familias de botnets IoT.** Investigado de forma concreta y no descartado por suposición, y aplazado deliberadamente por tres razones específicas, cada una verificada de forma independiente: el analizador ELF actual de la pipeline extrae solo metadatos estructurales básicos, sin ninguna regla de detección ATT&CK conectada, a diferencia de la vía PE con sus aproximadamente 86 técnicas mapeadas; la propia documentación de CAPE describe su soporte de sandboxing Linux/ELF como una capacidad en desarrollo activo, no de nivel producción; y todavía no existe ninguna muestra de familia ELF con datos de referencia frente a la que verificar cualquier nueva regla de detección específica de Linux, sin lo cual construir reglas repetiría exactamente el riesgo de sobreajuste que ya señala §6 para el lado Windows. Dado lo desproporcionadamente comunes que son las botnets basadas en ELF en dispositivos IoT y Linux embebido en comparación con la atención que les presta la propia literatura de investigación de malware, se considera una extensión genuinamente valiosa, pero una que primero necesita una muestra de familia ELF validada y una capacidad real de análisis dinámico, no un conjunto de reglas reajustado sobre el análisis estático por sí solo.
- **Un estudio de calibración de confianza con potencia estadística.** El hallazgo de §4.4 de que 16 de las 17 discrepancias auditadas provienen de hallazgos de una sola fuente, con la única excepción entre fuentes (T1497 de RoningLoader) atribuible a una cuestión de completitud de los datos de referencia y no a un nivel de confianza mal calibrado, es una señal direccional de tres muestras, no una afirmación con potencia estadística, y se beneficiaría de un conjunto de validación más grande y diseñado para ello.
- **Recuperación de claves en tiempo de ejecución mediante emulación.** La clave RC4 que nunca existe como bytes contiguos en la DLL de RoningLoader (§6, punto 4) es, en principio, recuperable emulando su rutina de ensamblado (p. ej. con Unicorn) en lugar de mediante cualquier búsqueda de patrón de bytes estática, un esfuerzo de ingeniería sustancialmente mayor que el módulo actual de recuperación estática de configuración, y fuera de alcance de esta tesis en consecuencia.

8. Conclusión

Esta tesis ha presentado MalWhere, una pipeline que combina análisis estático y dinámico de malware para producir inteligencia de amenazas con confianza cuantificada y conforme a estándares: mapeos de técnicas MITRE ATT&CK exportados como paquetes STIX 2.1 y eventos MISP en vivo. Dos mecanismos la extienden más allá de un diseño estático-más-dinámico convencional: un modelo de reconciliación de confianza que reconoce concordancia tanto entre fuentes como entre niveles de especificidad de técnica (§3.4), y un bucle de reenvío que atribuye correctamente la capacidad distribuida de una muestra multietapa al componente que realmente la presenta, en lugar de colapsarla en la puntuación propia del binario padre (§3.5). Frente a las herramientas revisadas en §2, la pipeline no pretende sustituir a un agregador de reputación amplio como VirusTotal, al que ninguna pipeline de investigación de un solo equipo puede igualar en escala bruta de corpus de envíos; es el paso que viene después, convirtiendo esa primera señal en un mapeo reconciliado, por técnica y ligado a evidencia para la muestra concreta que realmente lo necesita.

Validado frente a tres familias de malware con arquitecturas sustancialmente distintas (un binario de ransomware de una sola etapa, un infostealer .NET rico en funcionalidades, y una cadena de cuatro componentes cargador/rootkit/RAT) contra datos de referencia contruidos a partir de ingeniería inversa manual a nivel de función, la pipeline alcanzó puntuaciones F1 a nivel de familia de 0.93, 0.81 y 0.89 respectivamente, con precisión alta en todos los casos (§4). Esas cifras solo son tan fiables como el proceso que las produjo, razón por la cual esta tesis otorga igual peso al rastro de auditoría que las respalda: 17 discrepancias entre el resultado de la pipeline y los datos de referencia se rastrearon individualmente hasta una causa concreta y verificable, no se aceptaron ni descartaron solo por la cifra agregada (§4.3); se encontró y corrigió un error real de agregación que había sustituido silenciosamente el binario analizado incorrecto (§5.2); y la cobertura de detección estática se amplió de unas 30 a aproximadamente 86 identificadores de técnica MITRE, cada adición obtenida de las propias descripciones de técnica de MITRE y confirmada para no introducir nuevos hallazgos en el conjunto validado antes de confiar en ella (§3.6).

El argumento central de este trabajo es que este rastro de auditoría, cada regla ligada a evidencia verificable, cada discrepancia diagnosticada a una causa concreta, cada extensión comprobada frente a regresiones antes de conservarse, es una contribución más defendible que las cifras F1 destacadas por sí solas, y es la propiedad con mayor probabilidad de transferirse cuando la pipeline se encuentre con una familia de malware para la que no ha sido afinada. Precisamente porque esa transferencia todavía no se ha medido, ampliar el conjunto de familias validadas (§7) se identifica como el siguiente paso de mayor valor, junto con la extensión concretamente acotada al análisis de botnets IoT basadas en Linux/ELF una vez que sus dos prerequisites faltantes, reglas de detección realmente conscientes de ELF y una muestra Linux con datos de referencia, estén disponibles.

Referencias

- [1] MITRE Corporation, “MITRE ATT&CK.” <https://attack.mitre.org/>, 2026. Enterprise Matrix, accessed 2026.
- [2] MISP Project, “MISP: Open Source Threat Intelligence Platform.” <https://www.misp-project.org/>, 2026.
- [3] CAPE Sandbox Project, “CAPE: Config And Payload Extraction.” <https://github.com/kevoreilly/CAPEv2>, 2026.
- [4] Cuckoo Sandbox Project, “Cuckoo Sandbox: Automated Malware Analysis.” <https://cuckoosandbox.org/>, 2022.
- [5] Mandiant FLARE Team, “capa: The FLARE Team’s Open Source Tool to Identify Capabilities in Executable Files.” <https://github.com/mandiant/capa>, 2026.
- [6] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu, and X. Niu, “TTPDrill: Automatic and Accurate Extraction of Threat Actions from Unstructured Text of CTI Sources,” in *Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC)*, 2017.
- [7] MITRE Engenuity Center for Threat-Informed Defense, “TRAM: Threat Report ATT&CK Mapper.” <https://github.com/center-for-threat-informed-defense/tram>, 2026.
- [8] V. Legoy, M. Caselli, C. Seifert, and A. Peter, “Automated Retrieval of ATT&CK Tactics and Techniques for Cyber Threat Reports.” <https://arxiv.org/abs/2004.14322>, 2020. arXiv:2004.14322.
- [9] VirusTotal (Google), “VirusTotal.” <https://www.virustotal.com/>, 2026.
- [10] OASIS Cyber Threat Intelligence Technical Committee, “STIX Version 2.1.” <https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html>, 2021.
- [11] Mandiant FLARE Team, “FLOSS: FLARE Obfuscated String Solver.” <https://github.com/mandiant/flare-floss>, 2026.
- [12] MITRE Corporation, “MITRE ATT&CK STIX Data (Enterprise).” <https://github.com/mitre/cti>, 2026. `enterprise-attack.json`, accessed 2026.

A. Ampliación de cobertura de reglas de detección

Listado completo, técnica por técnica, de los seis lotes de cobertura resumidos en §3.6. Cada fila se verificó tras añadirse para no activarse en ninguna de las tres muestras validadas, confirmando que la ampliación no altera la Tabla 4.

Técnica	Nombre	Mecanismo de detección
Lote 1: Persistence y Defense Evasion		
T1546.010	AppInit DLLs	cadena de registro AppInit_DLLs
T1547.004	Winlogon Helper DLL	cadena Winlogon\Shell\Userinit
T1547.005	Security Support Provider	cadena Security Packages
T1547.002	Authentication Package	cadena Authentication Packages
T1546.012	Image File Execution Options Injection	cadena Image File Execution Options
T1546.003	WMI Event Subscription	cadena __EventFilter/ __EventConsumer/ __FilterToConsumerBinding
T1197	BITS Jobs	cadena bitsadmin
T1685.005	Clear Windows Event Logs	cadena wevtutil cl
T1218.010	Regsvr32 (Squiblydoo)	cadena scrobj.dll
T1218.005	Mshta	mshta.exe + .hta (ambos requeridos)
T1055.012	Process Hollowing	combinación (§3.2, Tabla 1)
Lote 2: Credential Access y Collection		
T1003.001	LSASS Memory	importación MiniDumpWriteDump + cadena lsass.exe
T1003.002	Security Account Manager	cadena hklm\sam
T1003.004	LSA Secrets	cadena SECURITY\Policy\Secrets
T1552.002	Credentials in Registry	patrones de ruta de credenciales en registro
T1552.004	Private Keys	marcadores PEM/OpenSSH, .ppk
T1555.004	Windows Credential Manager	importación VaultOpenVault, cadena VaultCli.dll
T1556.002	Password Filter DLL	cadena Notification Packages
T1040	Network Sniffing	cadena wpcap.dll/Packet.dll
T1125	Video Capture	cadena avicap32.dll
T1114.001	Local Email Collection	cadena .ost/.pst
T1039	Data from Network Shared Drive	WNetOpenEnumW/A + WNetEnumResourceW/A + WNetCloseEnum
Lote 3: Execution y Command and Control		
T1053.005	Scheduled Task	cadena schtasks / patrón de comando
T1059.001	PowerShell (codificado/oculto)	-EncodedCommand + -WindowStyle Hidden (am- bos requeridos)
T1559.002	Dynamic Data Exchange	cadena DDEAUTO
T1127.001	MSBuild	cadena msbuild.exe
T1569.002	Service Execution	cadena sc start/net start
T1102.002	Web Service (bidireccional)	api.telegram.org, pastebin.com/raw, raw.githubusercontent.com, discord.com/api/webhooks
T1572	Protocol Tunneling	Renci.SshNet, ngrok, serveo.net
T1090.002	External Proxy	cadena :9050 (puerto SOCKS de Tor)
Lote 4: Discovery		
T1518.001	Security Software Discovery	cadena MsMpEng.exe / avp.exe / SecurityCenter2
T1049	System Network Connections Discovery	importaciones GetTcpTable / GetExtendedTcpTable / GetUdpTable
T1087.001	Local Account Discovery	importación NetUserEnum
T1018	Remote System Discovery	importación NetServerEnum
T1120	Peripheral Device Discovery	importación SetupDiGetClassDevsw/A
T1614.001	System Language Discovery	importaciones GetSystemDefaultLangID/ GetUserDefaultUILanguage
T1010	Application Window Discovery	combinación (§3.2, Tabla 1)
T1033	System Owner/User Discovery	importación GetUserNameW/A (trasladada desde T1082)
Lote 5: Impact		
T1529	System Shutdown/Reboot	importación ExitWindowsEx
T1531	Account Access Removal	importación NetUserChangePassword
T1561.001	Disk Content Wipe	cadena \\.\PhysicalDrive
T1496	Resource Hijacking	cadena stratum+tcp://
Lote 6: Privilege Escalation y Exfiltration		
T1134.001	Token Impersonation/Theft	ImpersonateLoggedOnUser + DuplicateToken(Ex)
T1134.002	Create Process with Token	importación CreateProcessWithTokenW

continúa en la página siguiente

Técnica	Nombre	Mecanismo de detección
T1134.003	Make and Impersonate Token	<code>ImpersonateLoggedOnUser + LogonUserW/A</code>
T1548.002	Bypass User Account Control	cadena <code>ms-settings \ Shell \ Open \ command</code>
T1546.008	Accessibility Features	cadena <code>sethc.exe</code>
T1546.009	AppCert DLLs	cadena <code>AppCertDlls</code>
T1547.014	Active Setup	cadena <code>Active Setup\Installed Components</code>
T1098	Account Manipulation	cadena <code>net localgroup administrators</code>
T1567.001	Exfil. a repositorio de código	cadena <code>api.github.com</code>
T1567.002	Exfil. a almacenamiento en la nube	<code>dropboxapi.com / storage.googleapis.com</code>
T1567.003	Exfil. a sitios de almacenamiento de texto	<code>pastebin.com / api / api_post.php</code>
T1567.004	Exfil. por webhook	mapeo dual desde la evidencia de webhook de Discord de T1102.002
T1048.003	Exfil. por protocolo no cifrado ajeno a C2	importación <code>FtpPutFileW/A</code>

Cuadro 5: 55 identificadores de técnica añadidos a lo largo de seis lotes de cobertura (§3.6), obtenidos de las propias descripciones de técnica de MITRE.